

SAP DOCS · FIELD GUIDE

SAP GUI

The Complete Tour

Every bar, button, tab and screen in the S/4HANA SAP GUI

"The GUI is not the hard part of SAP. But being slow in it makes everything else look hard — and in an interview, hesitation at the command field reads as inexperience before you have written a line of ABAP."

8

PARTS

35

TOPICS

60+

T-CODES

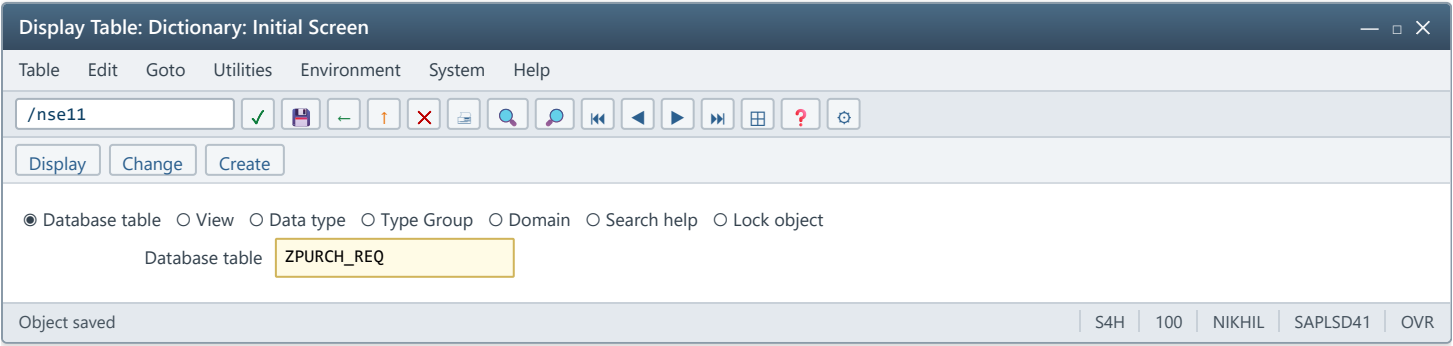
Table of Contents

- SAP GUI — The Complete Tour
- How to Use This Guide
- Part A — What SAP GUI Actually Is
 - A1. SAP GUI in One Picture · A2. The Three Flavours of SAP GUI · A3. The Logon Pad — Getting In · A4. GUI, Fiori and Eclipse — Who Does What in S/4HANA
- Part B — The Window, Strip by Strip
 - B1. The Six Strips Every Screen Has · B2. The Menu Bar — Including the Two That Are Always There · B3. The Standard Toolbar — Every Button · B4. The Application Toolbar and the Screen Body · B5. The Status Bar — Reading What SAP Is Telling You
- Part C — Moving Around
 - C1. The Command Field — Your Fastest Tool · C2. SAP Easy Access — The Home Screen · C3. Sessions — Working in Two Places at Once · C4. Finding a Transaction Code You Do Not Know · C5. Making SAP GUI Yours — Options and Themes
- Part D — The Transaction Code Reference
 - D1. Development and the ABAP Workbench · D2. Data — Looking At It and Maintaining It · D3. When Something Breaks — Debugging and Monitoring · D4. Transports, Users, and the Business Transactions You Will Meet
- Part E — Creating Your First ABAP Program
 - E1. SE38 and the Program Attributes Screen · E2. Packages, and Why \$TMP Matters · E3. Transport Requests — How Code Leaves Your System · E4. Write, Check, Activate, Run
- Part F — SE80 and What Moved to Eclipse
 - F1. SE80 — The Object Navigator · F2. SE80 vs Eclipse (ADT) — the Line You Must Know
- Part G — The Data Model and SE11
 - G1. The SAP Data Model in One Picture · G2. The SE11 Initial Screen — Every Option · G3. Domains — The Technical Layer · G4. Data Elements — The Meaning Layer · G5. Creating a Z Table, Step by Step · G6. Views, Search Helps and Lock Objects
- Part H — Reading Data, Debugging, and Your First Week
 - H1. SE16N — Reading Table Data Properly · H2. SM30 and the Table Maintenance Generator · H3. The ABAP Debugger · H4. Your First Week, and the Keys Worth Memorising

How to Use This Guide

*This is a **map**, not a course. Keep it open beside your 9–11 practice slot: when you land on a screen you do not recognise, find it here, and the callouts will tell you what every button on it does. Parts A–C are the ones to read straight through — they cover the window itself, which never changes no matter which transaction you are in. Everything after that is reference.*

The one thing to understand before anything else: SAP GUI is a **shell**. Every transaction — SE11, SE38, VA03, ST22 — runs inside the *same* window, with the same title bar, the same toolbar and the same status bar. Only the middle changes. Once you know the frame, you are never fully lost in a screen you have not seen before.



The SAP GUI frame, drawn to scale. **Every** transaction you ever open sits inside this same shell — title bar, menu bar, standard toolbar, application toolbar, screen body, status bar. Part B takes each strip apart.

Telugu: Idi course kaadu, **map**. Nee 9–11 practice slot pakkana ee doc pettuko — teliyani screen vachinappudu ikkada vetuku. Mukhyamaina vishayam modate ardhham chesuko: **SAP GUI oka shell (frame) matrame**. SE11 aina, SE38 aina, VA03 aina — anni **ide window lo** nadustayi. Paina title bar, menu, toolbar; kinda status bar — ivi eppudu marav. **Madhya bhagam matrame** marutundi. Ee frame telisthe, kotta screen vachina nuvvu tappipovu.

How this guide is organised:

Part	Covers	Read it
A	What SAP GUI is, the flavours, the Logon Pad, and where GUI sits next to Fiori and Eclipse	Straight through
B	The window taken apart strip by strip — every toolbar button, every status field	Straight through
C	Moving around — the command field, Easy Access, sessions, finding a T-code, GUI settings	Straight through
D	The transaction-code reference, grouped by what you are trying to do	Reference
E	Creating your first ABAP program in SE38 — packages and transports	When you do it
F	SE80 Object Navigator, and what moved to Eclipse in S/4HANA	Reference
G	The data model and SE11 — domains, data elements, and building a Z table end to end	When you do it
H	Reading data, and what to open when something breaks	Reference

Part A — What SAP GUI Actually Is

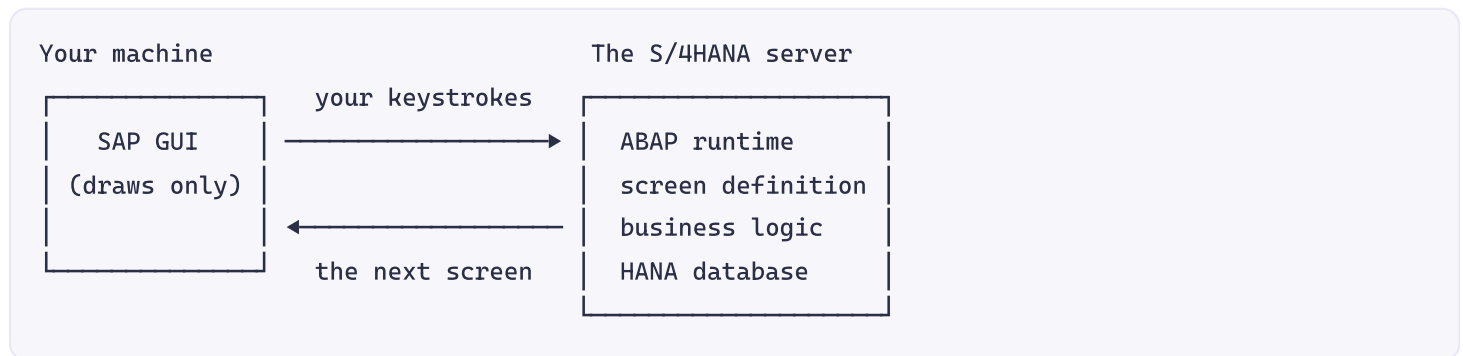
A1. SAP GUI in One Picture

Simple definition: **SAP GUI** (Graphical User Interface) is the desktop program you install on your machine to log into an SAP system. It draws the screens, sends your keystrokes to the server, and displays whatever comes back.

Telugu: SAP GUI ante nee computer lo install chese **oka desktop program**. Idi SAP system loki login avvadaniki. Nuvvu type chesindi server ki pampistundi, server icchina jawab ni screen meeda chupistundi. **Mukhyam:** SAP GUI lo **emi ledu** — antha server lo undi. Idi kevalam oka **kitiki (window)** matrame.

The critical mental model — and it is different from what you are used to: SAP GUI holds almost no logic. It is a *thin client*. Every screen you see was defined on the server, sent down as a description, and painted by the GUI. When you press Enter, your input goes back to the server, ABAP runs, and a new screen comes down.

Anchor to what you know: this is closer to a **server-rendered PHP page** than to a React app. In React, logic lives in the browser and you call an API for data. In SAP GUI, the "browser" is nearly empty — the server decides everything, including what the screen looks like. That is why SAP GUI feels like a page reload on every Enter: because it *is* one.



Why this matters practically: when a screen is slow, it is almost never the GUI — it is the server or your network. And when you cannot find a button, it is because the *server* did not send it, usually because of your authorizations or the mode you are in (display vs change).

Real-world: SAP GUI has looked broadly the same since the 1990s. That is not neglect — it is deliberate. Tens of thousands of transactions across every SAP module rely on the same frame, so SAP changes it very slowly. For you that stability is good news: what you learn this week still applies in 2035.

A2. The Three Flavours of SAP GUI

Simple definition: SAP ships three different SAP GUI programs. They connect to the same system but differ in features and in what they run on.

Telugu: SAP GUI **moodu rakalu** untayi. Anni oke system ki connect avutayi, kaani features veru. Nuvvu Windows lo **SAP GUI for Windows** vaadutavu — adi anni features unnadi, industry lo standard adhe.

Flavour	Runs on	Use it when	Limitations
SAP GUI for Windows	Windows only	This is the one you will use. Full feature set, all transactions, scripting, GuiXT	Windows only
SAP GUI for Java	Windows, macOS, Linux	You are on a Mac or Linux machine	Fewer features; some controls render differently
SAP GUI for HTML (Web GUI)	Any browser, via ITS	Quick access with nothing installed; often used for occasional users	Noticeably limited; some screens behave oddly

What you should have: SAP GUI for Windows, a recent release (SAP GUI 7.70 or 8.x for S/4HANA landscapes). Your institute will have told you which one — if you have it running in the 9–11 slot, you have the right one.

Themes matter more than you expect. SAP GUI 7.70+ ships several visual themes:

Theme	Look
SAP Signature	The classic grey-blue look most screenshots show
Belize / Belize Deep	Flatter, closer to Fiori's colours
Quartz Light / Dark (GUI 8.x)	The newest, matching modern Fiori

Your screens may look different in colour from this guide's mockups. **The layout is identical** — only the paint changes. Do not let a colour difference make you think you are on a different screen.

A3. The Logon Pad — Getting In

Simple definition: **SAP Logon** is a separate small program that holds your list of systems. You pick one, and it launches SAP GUI connected to it.

Telugu: SAP Logon (Logon Pad) ante — nee systems list unna oka chinna program. Andulo nunchi oka system select chesi double-click cheste, SAP GUI open avutundi. Ikkada **Client, User, Password, Language** adugutundi — ee naalugu lo **Client** ye chala mandi ki kotta vishayam.

The logon screen has four fields. Three are obvious. One is not:

The SAP logon screen. 1 Client — the one that confuses newcomers. 2 Your user ID. 3 Password. 4 Language, almost always EN.

Client — the concept with no web equivalent. A **client** is a completely separate set of business data inside the *same* SAP system. Client 100 and client 200 share the same programs and the same database, but their business data is invisible to each other.

Analogy: one WordPress installation serving two completely separate websites from the same database, where every table has a `site_id` column and every query silently filters on it. In SAP that column is called **MANDT**, it is the first key field of nearly every table, and the system adds the filter for you automatically.

Typical client	Purpose
000	SAP's master client — do not touch

Typical client	Purpose
100 / 200 / 800	Working clients — your practice client is one of these
Training systems	Often give each student a separate user in one shared client

Practical consequences you will hit:

- Create a table entry in client 100 and it does *not* exist in client 200. Same system, different data.
- **Programs, tables and DDIC objects are client-independent** — they are shared across all clients. Create a Z table in client 100 and its *definition* exists everywhere; only its *rows* are per-client.
- If a colleague says "I can't see your data", the first question is always "which client are you in?"

Know your client and system at all times. Both are shown in the status bar (Part B6), and [System → Status](#) gives you the full detail — including the S/4HANA release, which is a question you will be asked in interviews.

A4. GUI, Fiori and Eclipse — Who Does What in S/4HANA

Simple definition: S/4HANA has three front ends, and each one owns a different job. Knowing which is which prevents a lot of wasted searching.

Telugu: S/4HANA lo **moodu** front-ends unnayi — SAP GUI, Fiori, inka Eclipse (ADT). Prathi dhaaniki vere pani undi. Idi telithe "ee vishayam ekkada cheyyali?" ane confusion poutundi. **Chala mukhyamainadi:** CDS views inka RAP ni **SAP GUI lo cheyyalemu** — avi Eclipse lo matrame.

	SAP GUI	Fiori Launchpad	Eclipse + ADT
Who uses it	Developers, consultants, power users	Business end users	Developers only
Looks like	Dense grey screens	Modern web tiles	An IDE
Used for	Classical development, config, admin, debugging	Doing daily business work	Modern ABAP development
Your use	Daily, in the 9–11 slot	Later, in the Fiori track	From October, and it matters

The dividing line that trips people up:

Object	SAP GUI (SE11/SE80)	Eclipse (ADT)
Tables, domains, data elements	✓ Create and edit	✓ Create and edit
Classical reports, function modules, classes	✓ Create and edit	✓ Create and edit
CDS views	✗ Display only	✓ Only place to create them
RAP behaviour definitions	✗ Not at all	✓ Only place
ABAP Cloud development	✗ Not at all	✓ Only place

This is the single most important line in Part A. Your December portfolio app is CDS → RAP → OData → Fiori. **None of the CDS or RAP work can be done in SAP GUI.** It requires Eclipse with ABAP Development Tools, pointed at a system that supports it.

So SAP GUI is where you learn the fundamentals and where you will spend your first months on a real project — reading other people's classical code, checking tables, chasing dumps. But it is not where the modern half of your target stack gets built.

Analogy: SAP GUI is the workshop; ADT is the CAD station. You still go to the workshop every day, but the new designs are not drawn there.

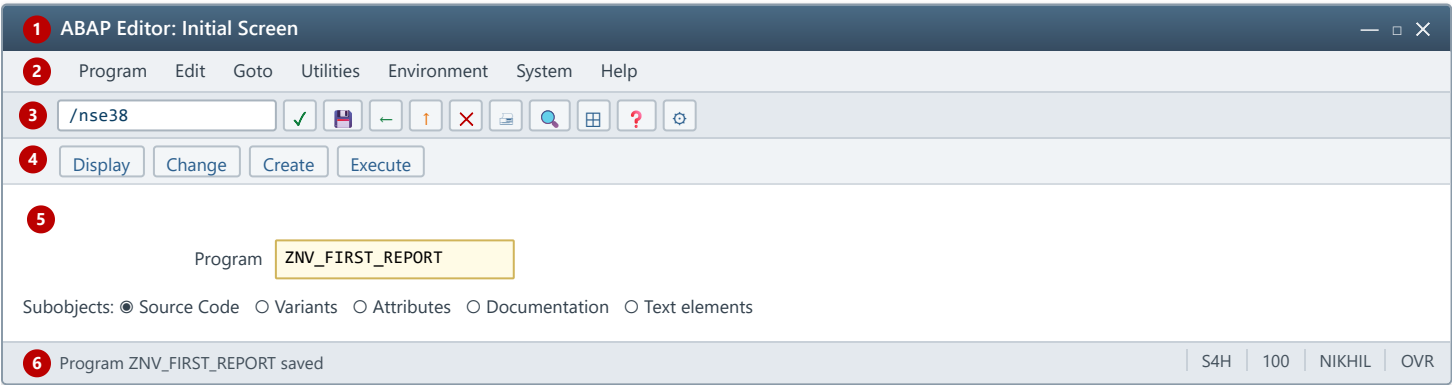
Action from this section: ask your FLT instructor whether your practice system supports CDS and RAP, and whether they will cover ADT. If the answer is no to either, that gap is yours to close — and it is the highest-priority gap you have.

Part B — The Window, Strip by Strip

B1. The Six Strips Every Screen Has

Simple definition: every SAP GUI screen is built from the same six horizontal strips, stacked top to bottom. Learn the six and you can orient yourself on any transaction in SAP.

Telugu: Prathi SAP screen lo **aaru pattilu (strips)** untayi, paina nunchi kinda varaku. Ee aaru telisthe, e transaction open chesina nuvvu ekkada emi undo cheppagalavu. Madhya unna **screen body** matrame prathi transaction ki maarutundi — migilina aidu eppudu okate.



SE38, with the six strips numbered. 1 Title bar 2 Menu bar 3 Standard toolbar + command field 4 Application toolbar 5 Screen body 6 Status bar.

#	Strip	Changes per transaction?	What it is for
1	Title bar	Text changes	Tells you which screen you are on
2	Menu bar	First menus change; System and Help never do	Every function the screen offers

#	Strip	Changes per transaction?	What it is for
3	Standard toolbar	Never changes	Command field + the universal buttons (save, back, exit...)
4	Application toolbar	Always changes	The most-used buttons <i>for this transaction</i>
5	Screen body	Always changes	The actual work area
6	Status bar	Message changes; fields do not	Messages, plus system/client/user/program

The practical rule: if you are lost, **strips 3 and 6 are always the same**. The command field can take you anywhere and the status bar always tells you where you are. Those two are your anchors.

Anchor to what you know: strips 1–3 are the browser chrome — the title bar and address bar that never change. Strip 5 is the page content. Strip 4 is the page's own toolbar. Strip 6 is the browser status bar, except SAP's actually tells you something useful.

B2. The Menu Bar — Including the Two That Are Always There

Simple definition: the menu bar holds every function the current screen offers. The first few menus change per transaction; the last two — **System** and **Help** — are identical everywhere in SAP.

Telugu: Menu bar lo aa screen lo unna **anni functions** untayi. Modati konni menus prathi transaction ki maarutayi, kaani **System** inka **Help** — ee rendu SAP antha okatela untayi. Application toolbar lo kanapadani options chala varaku ikkada untayi, anduke button dorakakapote **menu lo vetuku**.

The typical shape: <Object> | Edit | Goto | Utilities | Environment | System | Help

Menu	Usually contains
First menu (Program, Table, List...)	Create, change, display, activate, check, print, exit — named after the object you are working on
Edit	Cut, copy, paste, select, delete, insert lines
Goto	Jump to another part of the <i>same</i> object — attributes, technical settings, documentation
Utilities	Tools around the object — where the Table Maintenance Generator hides in SE11
Environment	Related objects elsewhere — where cross-references and usage lists live

System — the menu worth memorising. This one is available on every single screen:

System →	What it does	Why you care
Create Session	Opens a second window into the same system	Work in two transactions at once
End Session	Closes the current window	

System →	What it does	Why you care
User Profile → Own Data	Your user settings, default printer, date format, parameter IDs	Set date format to DD.MM.YYYY once and forget it
User Profile → Hold/Set Data	Pre-fill screen fields for next time	Saves retyping the same test data
Services → Table Maintenance	Jump straight to SM30	
Services → Reporting	Jump straight to SE38	
Services → ABAP Workbench	Jump to the dev tools	
List	Save, print or download the current list output	How you get an ALV out to a spreadsheet
Status...	The system information box	The most useful item in this menu — see below
Log off	Leave the system	

System → Status is the one to know cold. It opens a dialog telling you:

- **Client, User, Language** — where you are
- **Transaction** and **Program** — what is currently running
- **SAP release / component version** — "S/4HANA 2023" — the answer to a very common interview question
- **Database system** and **host** — HANA, and which server

Interview relevance: "Which S/4HANA release did you work on?" is asked constantly. **System → Status** is where the answer lives. Look it up in your practice system this week and remember the number.

Help → the F1 route. **Help → Application Help** opens SAP's documentation for the current screen. Useful, though in practice you will press **F1** on a field far more often (Part B4).

B3. The Standard Toolbar — Every Button

Simple definition: the row of small icons under the menu bar. It is **identical on every screen in SAP**, which makes it the highest-value thing in this guide to learn properly.

Telugu: Ee toolbar **SAP antha okate** — e screen lo aina ivi ee buttons ye untayi. Anduke deenini **gattiga nerchuko**, oke sari nerchukunte SAP motham lo pani chestundi. Mukhyanga **Back, Exit, Cancel** — ee moodintiki teda telisi undali, kotta vaallu ikkade tappu chestaru.

Icon	Name	Key	What it does
	Command field	Ctrl+/ 	Type a T-code to go anywhere (Part C1)

Icon	Name	Key	What it does
✓	Enter / Continue	Enter	Confirm input and let the server process the screen. Not "save".
	Save	Ctrl+S	Commit your work to the database
←	Back	F3	Go back one screen
↑	Exit	Shift+F3	Leave the transaction completely
×	Cancel	F12	Abandon the current screen without processing
	Print	Ctrl+P	Send the current list to spool
	Find	Ctrl+F	Search within the current list or code
	Find next	Ctrl+G	Repeat the last search
⏮ ⏪ ⏩ ⏭	First / Previous / Next / Last page	Ctrl+PgUp, PgUp, PgDn, Ctrl+PgDn	Scroll long lists
	Create new session	—	Opens another window into the same system
	Create shortcut	—	Saves a desktop shortcut that opens straight into this transaction
?	Help	F1	Context help for the field the cursor is in
	Customize local layout	Alt+F12	GUI settings, themes, clipboard, scripting (Part C5)

Back vs Exit vs Cancel — the distinction that catches everyone:

Button	Key	Behaviour	Use when
Back	F3	One screen back, staying in the transaction	You went one level too deep
Exit	Shift+F3	Leaves the transaction entirely, back to Easy Access	You are done here
Cancel	F12	Discards the current screen's input without processing	You typed something wrong and want out

The mistake to avoid: ✓ (Enter) is **not** Save. Enter tells the server "process what I typed" — it validates, fills in derived fields, and moves you on. Your work is only in the database after **Ctrl+S**. In an editor, you additionally need **Activate** (Ctrl+F3) before anything actually runs.

Anchor to what you know: Enter is like submitting a form for validation. Save is like the POST that writes to the database. Activate is like a build step — the object exists but is not live until compiled. Three separate actions; web development usually collapses them into one.

B4. The Application Toolbar and the Screen Body

Simple definition: the application toolbar holds the buttons specific to *this* transaction. The screen body is where you actually work — and two function keys inside it will save you more time than anything else in this guide.

Telugu: Application toolbar lo aa transaction ki matrame sambandhinchina buttons untayi — anduke idi prathi screen ki maarutundi. Screen body lo nuvvu pani chestavu. Ikkada **rendu keys** chala mukhyam: **F1** (ee field ante emiti?) inka **F4** (ee field lo em values pettochu?). Ee rendu ABAP developer ki roju vaade tools.

Application toolbar: whatever the transaction's designers thought you would press most — *Display / Change / Create / Execute* in SE38, *Display / Change / Create* in SE11. If a button you expect is missing, look in the menu bar; the toolbar only shows a subset.

Field types you will see in the screen body:

Element	Looks like	Note
Input field	White box	Type into it
Required field	Box with a ✓ or highlighted	Cannot proceed while empty
Display-only field	Grey, no border	Read-only in this mode
Checkbox / Radio button	☐ / ☐	Radio = pick exactly one
Tab strip	Row of tabs	Same screen, more pages of fields
Table control / ALV grid	Spreadsheet-like	Sortable, filterable, exportable

F1 and F4 — the two keys that make you fast

F4 — value help. Put the cursor in a field and press **F4** to see the valid entries. Any field with a small dropdown arrow supports it. This is how you find a plant code, a company code, a package name, without asking anyone.

F1 — field help, and the hidden trick. Press **F1** on a field and you get its documentation. But the real prize is the button inside that popup:

F1 → Technical Information

That shows you the **table name and field name** behind the screen field you are standing on.

Why this matters enormously to you as an ABAP developer: a functional consultant says "the delivery date is wrong on the purchase order". You open ME23N, click the field, press F1 → Technical Information, and it tells you the value lives in **EKET-EINDT**. Now you can write the SELECT. Without this trick you would be guessing table names or searching the internet.

Performance Assistant

Technical Information

Customizing

Delivery Date — Date on which the goods are to be delivered.

Field Data

Table Name	EKET
Field Name	EINDT
Data Element	EEIND
DE Supplement	0

S4H

100

F1 → **Technical Information**. This is how you find the table and field behind any screen field — the single most useful trick in SAP GUI for a developer.

Practise it this week: open any transaction with data on screen, press F1 on three different fields, and note the table names. You will start recognising **EKKO** , **EKPO** , **MARA** , **VBAK** — the tables every ABAP interview mentions.

B5. The Status Bar — Reading What SAP Is Telling You

Simple definition: the bottom strip. The left half shows the system's message to you; the right half shows where you are.

Telugu: Kinda unna patti. **Yeda vaipu** — system nee ki chepthunna message. **Kudi vaipu** — nuvvu ekkada unnavu (system, client, user, program). Message ki **rangu (colour)** chala mukhyam — akupacha ante ayindi, erupu ante avvaledu.

The left half — messages, and their four types:

Type	Colour	Meaning	What happens
S — Success	Green	It worked	Processing continues
I — Information	Yellow/blue	Just telling you something	Continues
W — Warning	Yellow	Something is odd, but allowed	Continues if you press Enter again
E — Error	Red	Invalid; fix it	Stops — you cannot proceed
A — Abend	Red	Fatal	Transaction terminates

Double-click a message to see its long text — which usually explains far more than the one line. Most beginners never discover this and stay stuck on a red message they could have solved in ten seconds.

The right half — the system fields. Click the small arrow to toggle which of these show:

Field	Example	What it tells you
System	S4H	Which SAP system you are logged into
Client	100	Which client — see A3

Field	Example	What it tells you
User	NIKHIL	Who you are logged in as
Program	SAPLSD41	The ABAP program currently running
Transaction	SE11	The T-code you are in
Response time	0.3 s	How long the last round trip took
OVR / INS	OVR	Overwrite or insert typing mode

Why the Program field is genuinely useful: when you are debugging or trying to understand a screen you did not write, the status bar tells you the program name for free. From there, SE38 or SE80 opens the source.

A habit worth building from day one: after every action, glance at the status bar before doing anything else. Green means proceed. Red means read it — and double-click it. Most "SAP is broken" moments are an unread red message at the bottom of the screen.

Part C — Moving Around

C1. The Command Field — Your Fastest Tool

Simple definition: the small white box at the top left. Type a transaction code into it and press Enter to jump anywhere in SAP, from anywhere in SAP.

Telugu: Paina yeda vaipu unna chinna tella box. Andulo transaction code type chesi Enter kotte, SAP lo ekkadiki aina veltavu. **Mouse tho menu lo vetakadam manesi ee box vaadu** — adhe vegam. Oke jagratha: nuvvu already oka transaction lo unte, code mundu **/n** pettali.

The syntax — memorise these six:

Type this	What happens
SE11	Opens SE11 — only works from the Easy Access screen
/nSE11	Opens SE11 in the current window , ending what you were doing. Works from anywhere.
/oSE11	Opens SE11 in a new window , keeping your current one. Works from anywhere.
/n	Cancels the current transaction and returns to Easy Access
/o	Shows the list of your open sessions
/i	Ends the current session (closes this window)

The rule that confuses everyone at first: from the Easy Access screen, plain **SE11** works. From *inside* a transaction, you must write **/nSE11** — otherwise SAP thinks you are typing data into the current screen. When in doubt, always type the **/n**. It works from everywhere.

Two more you will want:

Type this	What happens
<code>/nend</code>	Log off, with a confirmation prompt
<code>/nex</code>	Log off immediately, no prompt — unsaved work is lost
<code>/h</code>	Switch on the debugger, then run the next action

Be careful with `/nex`. It is fast and it is unforgiving. Many people type it out of habit and lose an hour of unsaved work. Use `/nend`.

`/h` is how you start debugging in classical SAP GUI: type `/h`, press Enter (the status bar confirms "Debugging switched on"), then trigger the action you want to inspect. The debugger opens at the first line of the next ABAP statement.

Two conveniences worth knowing:

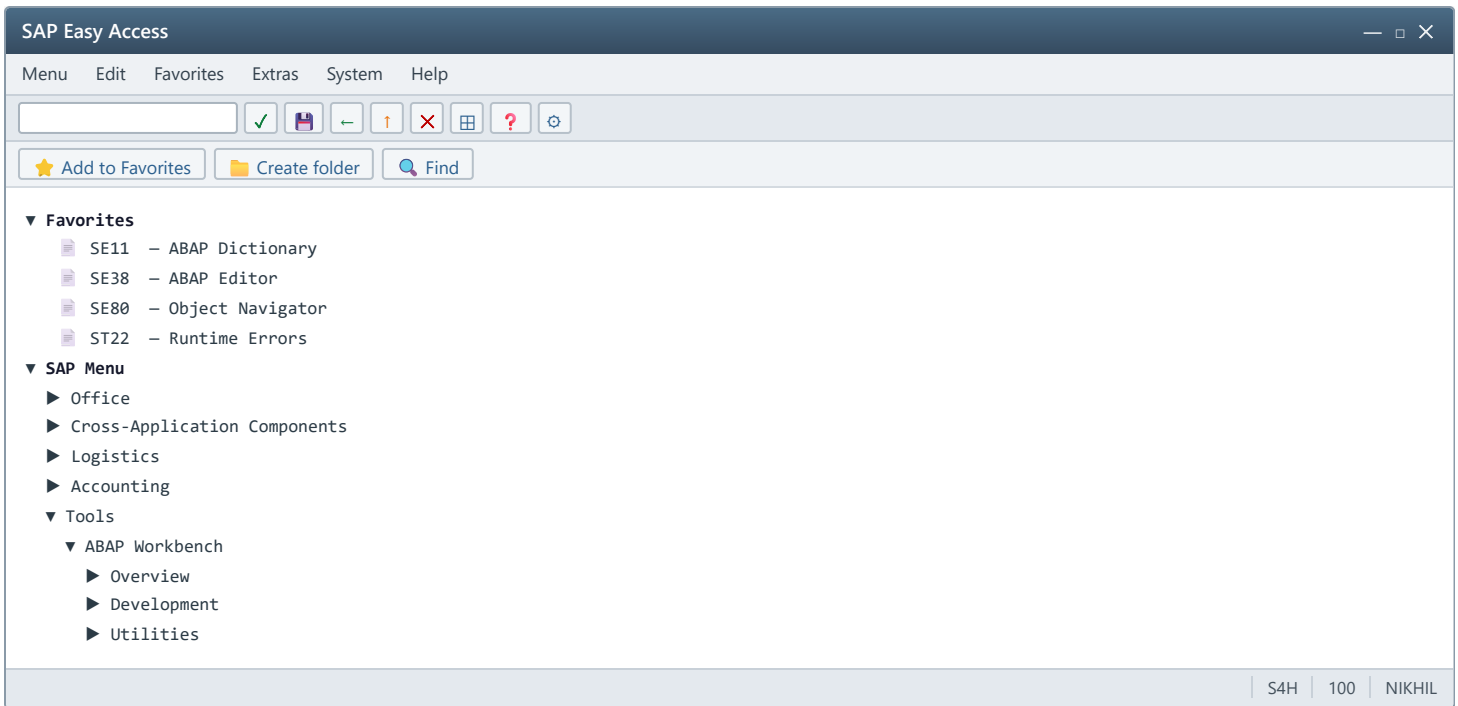
- The command field has a **dropdown history** — click the small arrow to see T-codes you used recently.
- **Ctrl+/ **puts the cursor straight into the command field from anywhere, so you never need the mouse.****

Anchor to what you know: the command field is the browser address bar, and T-codes are URLs. `/n` is "navigate in this tab", `/o` is "open in a new tab". Once you see it that way it stops feeling cryptic.

C2. SAP Easy Access — The Home Screen

Simple definition: the tree screen you land on after logging in. It is SAP's menu of every transaction you are allowed to run.

Telugu: Login ayyaka vachhe modati screen. Idi oka **tree menu** — SAP lo neeku permission unna anni transactions ikkada untayi. Kaani nija panilo evaru deenini vaadaru — andaru **command field lo T-code** type chestaru, endukante adi chala vegam. Kaani **Favorites** matram chala upayogam.



*SAP Easy Access with Favorites at the top. Nearly everything a developer needs sits under **Tools** → **ABAP Workbench** — but typing the T-code is far faster.*

The three trees:

Tree	What it holds
Favorites	Your own shortcuts — the only part most people actually use
SAP Menu	The complete standard menu, organised by business area
User Menu	Only what your assigned roles allow — often empty on a training system

Set up your Favorites in the first week. Right-click *Favorites* → *Insert transaction*, type the code, done. Add these ten and your daily work gets noticeably faster:

SE11 · SE38 · SE80 · SE24 · SE37 · SE16N · SM30 · ST22 · SE09 · SE93

You can also drag a transaction from the SAP Menu into Favorites, and group them into folders ([Favorites](#) → [Insert folder](#)).

Where developer transactions live in the SAP Menu, if you ever want to browse rather than type:

Tools	
└ ABAP Workbench	
└ Overview	→ SE80 Object Navigator, SE84 Info System
└ Development	→ SE38 Editor, SE11 Dictionary, SE24 Class Builder, SE37 Function Builder, SE91 Messages
└ Utilities	→ SE30/SAT Runtime Analysis, ST05 SQL Trace
└ Test	→ SE37 test, ABAP Unit

C3. Sessions — Working in Two Places at Once

Simple definition: a **session** is one SAP GUI window into the system. You can have several open at once, each in a different transaction, all under the same logon.

Telugu: Session ante oka SAP window. Nuvvu **okesari konni windows** teruchukovachu — okatilo table chusukuntu, inkokatilo code raayochu. Idi chala upayogam. Generally **aaru (6) varaku** allow chestaru. Browser lo tabs laage anukovachu.

How to open one:

Method

<code>/oSE11</code> in the command field	Opens SE11 in a new session directly
<code>/o</code> then pick from the list	Shows existing sessions and lets you create one
The  button on the standard toolbar	New session, same screen
<code>System → Create Session</code>	Same thing from the menu

The limit: most systems allow **six** sessions per user. It is a basis setting (`rdisp/max_alt_modes`), and when you hit it SAP simply refuses to open another.

Why this matters for you specifically: the single most common developer workflow is *two sessions side by side* — SE38 with your code in one, SE11 or SE16N with the table in the other. Being fluent with `/o` is a small thing that visibly separates people who use SAP daily from people who have only been taught it.

Ending sessions:

<code>/i</code>	Ends the current session
<code>System → End Session</code>	Same
<code>/nend</code>	Logs off all sessions, with confirmation

A warning about locks. If you are editing an object in one session, SAP locks it. Trying to open the same object in a second session gives you *"Object is locked by user NIKHIL"* — locked by *yourself*. It looks like a bug and it is not. Close the other session, or open the object in display mode.

C4. Finding a Transaction Code You Do Not Know

Simple definition: four ways to answer "what is the T-code for...?" without asking anyone.

Telugu: "Deeniki T-code emiti?" ani telisukovadaniki **naalugu daarulu**. Modati rendu roju vaade vi. Interviews lo kuda "T-code teliyakapote em chestavu?" ani adigithe, ee jawaabu manchi impression istundi.

Method 1 — `SEARCH_SAP_MENU` . Type it in the command field. It searches the whole SAP menu for a word and returns matching transactions with their menu paths.

Type `SEARCH_SAP_MENU` , enter `dictionary` , and it returns SE11 with the path `Tools → ABAP Workbench → Development → ABAP Dictionary` .

Method 2 — SE93, the Transaction Maintenance tool. SE93 is where transaction codes are defined. Put in a code to see exactly what it starts (program, screen, or method) — and use F4 with a wildcard to search:

In SE93, type `SE1*` and press F4 to list every transaction starting with SE1.

Method 3 — table TSTC via SE16N. Every transaction code in the system lives in table **TSTC**, and its text in **TSTCT**. Open SE16N, table `TSTCT` , filter `TTEXT` with `*dictionary*` . This is the developer's way and it is exhaustive.

Method 4 — the status bar. If someone shows you a screen and you want to know how they got there, look at the **Transaction** field in the status bar, or use `System → Status` . It names the T-code you are currently in.

The reverse question — "what does this T-code do?" Type it into SE93 and read the description, or simply run it in a *sandbox* client. Never explore unknown transactions in a production system; some of them post documents or start jobs.

Two naming patterns that help you guess:

Prefix	Area
SE	ABAP Workbench / development (SE11, SE38, SE80, SE93)
SM	System management / basis (SM30, SM37, SM59)
SU	User and authorization (SU01, SU53)
ST	System monitoring and trace (ST22, ST05)
Z / Y	Custom transactions built at your company

C5. Making SAP GUI Yours — Options and Themes

Simple definition: the  button (**Alt+F12**) opens SAP GUI's own settings. A handful of them are worth changing on day one; the rest you can ignore.

Telugu:  button (Alt+F12) — idi SAP GUI settings. Chala options unnayi, kaani **moodu matrame** ippude marchali: (1) prathi system ki vere **rangu** pettadam — production lo pai tappu cheyyakunda undadaniki, (2) dropdown lo **technical keys** chupinchadam — developer ki chala avasaram, (3) input **history** on cheyyadam.

The Alt+F12 menu gives you: *Options, New Visual Design, Clipboard, Character Set, Quick Cut and Paste, Activate GuiXT, Script Recording and Playback, Default Size, Hardcopy, Create Shortcut, About.*

Under Options, the tabs are:

Tab	Holds
Visual Design	Theme, fonts, colours in system , branding image
Interaction Design	Visualization, notifications, sounds, control settings
Accessibility & Scripting	Accessibility mode, GUI scripting on/off
Multilingual Settings	Language and I18N
Local Data	History , cache, configuration files
Traces	GUI and performance traces (basis use)
Security	Security rules for file access and scripting

The three settings to change today

1. Colour-code your systems. *Visual Design* → *Color Settings* → *Color in System*. Give each system a distinct colour — for example green for development, yellow for quality, **red for production**.

This is not decoration. It is the standard professional habit that stops someone running a test program in production because two windows looked identical. Interviewers notice when a candidate mentions it, because it signals project discipline rather than classroom learning.

2. Show technical keys in dropdowns. *Interaction Design* → *Visualization 1* → "Show keys within dropdown lists" (and "Sort by keys").

By default, a dropdown shows "*Standard Purchase Order*". With this on, it shows "*NB — Standard Purchase Order*". As an ABAP developer you need the key **NB**, because that is what is stored in the table and what your code will compare against. Leave this on permanently.

3. Turn on input history. *Local Data* → *History* → *On*. SAP GUI then autocompletes fields you have typed before, which on a system where you retype the same table names all day saves real time.

Worth knowing, not changing yet:

- **Quick Cut and Paste** — select text and it copies automatically. Convenient, but it can overwrite your clipboard unexpectedly.
- **Script Recording and Playback** — records your clicks into a script. Sometimes disabled for security. Useful much later for repetitive test data entry.
- **Default Size** — resets the window to SAP's standard dimensions.

Part D — The Transaction Code Reference

D1. Development and the ABAP Workbench

Simple definition: the transactions you will live in every morning. Learn the first six by heart; the rest you can look up.

Telugu: Ivi nuvvu roju vaade T-codes. **Modati aaru** — SE11, SE38, SE80, SE24, SE37, SE16N — ivi baaga gurthupettuko, roju kavali. Migilinaivi avasaram vachinappudu chusukovachu. Table lo ★ unnavi mukhyamainavi.

T-code	Name	What you use it for
SE11 ★	ABAP Dictionary	Tables, views, data elements, domains, search helps, lock objects
SE38 ★	ABAP Editor	Write, run and debug reports and programs
SE80 ★	Object Navigator	The all-in-one workbench — browse a package and edit anything in it
SE24 ★	Class Builder	Global classes and interfaces (OO ABAP)
SE37 ★	Function Builder	Function modules and function groups
SE93	Transaction Maintenance	Create a Z transaction code; look up what any T-code runs
SE91	Message Maintenance	Message classes — the texts your programs raise
SE84	Repository Information System	Search the whole repository by object type, author, package
SE39	Split-Screen Editor	Compare two programs side by side
SA38	ABAP Program Execution	Run a report by name without opening the editor
ABAPDOCU	ABAP Documentation & Examples	SAP's built-in tutorial with runnable example programs
SE18 / SE19	BAdI Definition / Implementation	Enhancements — extending standard SAP without modifying it
CMOD / SMOD	Customer enhancement projects	The older enhancement framework; still found in old systems
SE41 / SE51	Menu Painter / Screen Painter	Classical Dynpro UI — legacy, but you will read it
SE71 / SMARTFORMS	Form painters	Printed output — legacy, skim only

Start with ABAPDOCU this week. It is SAP's own documentation *with executable examples* built into the system. Every syntax topic has a demo program you can run and modify. For someone with 2 hours of system time a day, it is the cheapest practice material available and almost nobody at your stage knows it exists.

SE80 vs the individual transactions: SE80 can do almost everything SE38, SE11, SE24 and SE37 do, in one window with a navigation tree. Many developers live in SE80 alone. Learn both — SE80 for browsing and understanding a package, the individual T-codes for going straight to one object.

D2. Data — Looking At It and Maintaining It

Simple definition: the transactions for reading table contents and letting users maintain them.

Telugu: Table lo em data undo chudataniki, inka users ki data maintain cheyyaniche screens ki — ee T-codes. **SE16N** roju vaadutavu. Kaani **jagratha** — production system lo data chudatam parledu, kaani **marchadam** chala prammadam.

T-code	Name	Use
SE16N ★	General Table Display	The everyday data browser — flexible selection, easy export
SE16	Data Browser	The classic version; still everywhere
SE16H	HANA-optimised Table Display	Adds aggregation and grouping, S/4HANA only
SM30 ★	Table Maintenance	The screen end users get to maintain a Z table's contents
SM31	Table Maintenance (older)	Legacy equivalent
SM34	View Cluster Maintenance	Several related tables maintained together
SE54	Generate Table Maintenance Dialog	Creates the SM30 screen for your Z table
SE14	Database Utility	Adjust/activate the database object after a table change
SE13	Technical Settings	Buffering, size category — usually reached from inside SE11

The `&SAP_EDIT` warning. You will eventually hear that typing `&SAP_EDIT` in SE16N turns on direct table editing. Three things to know:

1. It bypasses all business logic and validation.
2. It is **logged and audited** on any well-run system.
3. On most production systems it is disabled outright.

Do not use it, and do not mention it as a solution in an interview. The correct answer to "how do I change this data?" is always *"through the transaction that owns it, or a maintenance dialog"*.

Exporting data you have selected: in SE16N or any ALV list, use `List → Export → Spreadsheet`, or the export icon on the ALV toolbar. This is how you get data out for analysis, and how you will build test data sets.

D3. When Something Breaks — Debugging and Monitoring

Simple definition: the transactions you open when a program fails, a job did not run, or a user says "it says I have no authorization".

Telugu: Emaina fail aithe, ee T-codes teruchali. **ST22** (dump vachinapudu), **SU53** (authorization problem), **SM37** (job run avvaledu) — ee moodu chala takkuva mandi ki telusu, kaani interviews lo **tappaka** adugutaru. "Production lo problem vasthe em chestavu?" ane prasnaki ivi ye jawaabu.

T-code	Name	Open it when
ST22 ★	ABAP Runtime Errors (Short Dumps)	A program crashed — this tells you the exact line and why

T-code	Name	Open it when
SU53 ★	Authorization Check	A user gets "no authorization" — shows the exact object that failed
SM37 ★	Job Overview	A background job did not produce its output
SM36	Define Background Job	Schedule a job
SM12	Lock Entries	"Object locked by another user" — see who holds it
SM13	Update Records	An update terminated after the user saw "saved"
SM21	System Log	System-level errors around a given time
SLG1	Application Log	Business-application logs written by programs
ST05	Performance / SQL Trace	Find the slow SQL statement
SAT (old SE30)	Runtime Analysis	Find where a program spends its time
SM50 / SM66	Work Process Overview	See what is running right now
SM59	RFC Destinations	Connections to other systems

How to read a short dump in ST22 — this is a genuinely useful skill and takes ten minutes to learn:

1. Open ST22, pick today, double-click the dump.
2. **Error analysis** — what went wrong in plain language.
3. **Source Code Extract** — the exact line, with the failing statement highlighted.
4. **Active Calls / Events** — the call stack: which program called which.
5. **Chosen variables** — what the variables contained at the moment of failure.

The most common dumps you will cause yourself while learning:

Dump	Usually means
<code>CX_SY_ZERODIVIDE</code>	Divided by zero
<code>CX_SY_CONVERSION_NO_NUMBER</code>	Tried to move text into a numeric field
<code>TABLE_INVALID_INDEX</code>	Read an internal table row that does not exist
<code>CX_SY_OPEN_SQL_DB</code>	A malformed SELECT
<code>DBIF_RSQ_L_INVALID_CURSOR</code>	Usually a nested SELECT gone wrong
<code>TIME_OUT</code>	A dialog step ran longer than the limit (default around 10 minutes)

Deliberately cause one this week. Write a two-line program that divides by zero, run it, then find your own dump in ST22 and read all five sections. Doing this once makes ST22 familiar rather than frightening — and "walk me through how you'd investigate a dump" is a stock interview question.

D4. Transports, Users, and the Business Transactions You Will Meet

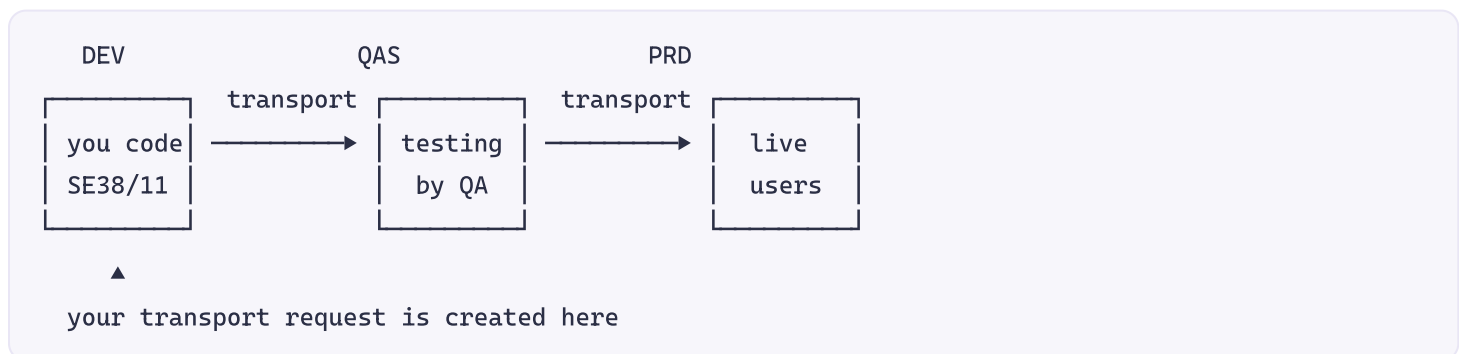
Simple definition: how your code moves between systems, who is allowed to do what, and the handful of business transactions an ABAP developer needs to recognise.

Telugu: Nee code DEV nunchi QA, PROD ki ela veltundo — adi **transport**. Inka nuvvu ABAP developer aina, business transactions konni telisi undali — endukante nuvvu vetike data avi create chestayi. **ME23N** tho purchase order chudu, F1 kotti table peru telusuko — appudu nee SELECT ela raayalo ardham avutundi.

Transports — moving your work:

T-code	Name	Use
SE09 ★	Transport Organizer	Your requests and tasks — release them from here
SE10	Transport Organizer	The same tool, different entry screen
SE01	Transport Organizer (extended)	Full view, including imports
SE03	Transport Organizer Tools	Search objects in requests, set system change options
STMS	Transport Management System	The basis view — imports into QA and production

The landscape your code travels through:



Users and authorizations:

T-code	Name	Use
SU01	User Maintenance	Create users, assign roles (usually basis, not you)
SU3	Own User Data	Your settings — date format, decimal notation, default printer
SU53	Authorization Check	The last failed check for a user
PFCG	Role Maintenance	Building authorization roles
SUIM	User Information System	Who has which authorization

Set your own defaults in SU3 today — date format **DD.MM.YYYY**, decimal notation, and time zone. It takes a minute and removes a small daily annoyance.

Business transactions worth recognising — you do not need to *use* these, but you need to know what data they create, because your reports will read it:

Area	Create / Change / Display	What it makes
Sales (SD)	VA01 / VA02 / VA03	Sales orders → tables <code>VBAK</code> , <code>VBAP</code>
Purchasing (MM)	ME21N / ME22N / ME23N	Purchase orders → <code>EKKO</code> , <code>EKPO</code> , <code>EKET</code>
Purchase requisition	ME51N / ME53N	Requisitions → <code>EBAN</code>
Material master	MM01 / MM02 / MM03	Materials → <code>MARA</code> , <code>MARC</code> , <code>MARD</code>
Business Partner	BP	Customers and vendors → <code>BUT000</code> , <code>KNA1</code> , <code>LFA1</code>
Goods movement	MIGO	Material documents → <code>MKPF</code> , <code>MSEG</code>
Billing	VF01 / VF03	Invoices → <code>VBRK</code> , <code>VBRP</code>
Accounting document	FB03	Journal entries → <code>ACDOCA</code> in S/4HANA
Stock overview	MMBE	Current stock by plant

Three S/4HANA changes that come up in interviews

Change	Detail
Business Partner is mandatory	Customer (XD01) and vendor (FK01) masters are replaced by the single BP transaction. <code>KNA1</code> and <code>LFA1</code> still exist underneath, but BP is the entry point.
The Universal Journal	Finance tables <code>BSEG</code> / <code>BKPF</code> and the old totals tables are consolidated into <code>ACDOCA</code> , one line-item table for all of finance.
Material number is 40 characters	Extended from 18. Old code assuming 18 breaks.

Knowing these three signals that you learned *S/4HANA*, not ECC — which is exactly the filter hiring managers are applying in 2026.

Part E — Creating Your First ABAP Program

E1. SE38 and the Program Attributes Screen

Simple definition: SE38 is the ABAP Editor. Creating a program there means naming it, filling in an attributes screen, then writing code.

Telugu: SE38 lo program create cheyyadam moodu adugulu: (1) peru ivvadam — **Z** tho modalu, (2) **Attributes** screen nimpadam, (3) code raayadam. Attributes screen lo chala fields untayi, kaani **rendu matrame** mukhyam — Title inka Type. Migilinavi default ye sari.

Step 1 — name it. Open SE38, type the name, click **Create**.

Program names must start with Z or Y. That is the customer namespace — everything SAP delivers starts with something else, so a Z prefix guarantees you never collide with, or accidentally modify, standard SAP code. Maximum 30 characters. A useful convention: `Z<initials>_<what it does>`, for example `ZNV_CARRIER_LIST`.

Step 2 — the attributes screen. It looks intimidating. Only two fields matter today.

ABAP: Program Attributes — ZNV_CARRIER_LIST — Create

✓

📁

←

✗

Title

List of airline carriers

1

Original language

EN

Type

Executable program

2

Status

Test program

Application

Authorization group

☒ Unicode checks active

☒ Fixed point arithmetic

S4H

100

The Program Attributes screen. **1 Title** — required, and it is what appears in every program list. **2 Type** — "Executable program" for a report. Leave the rest at their defaults.

The Type field, decoded:

Type	What it is	You will use it
Executable program (Type 1)	A report you can run with F8	Now — this is your default
Include program (I)	A code fragment pulled into another program	Later
Module pool (M)	Classical screen program (Dynpro)	Legacy
Function group (F)	Container for function modules	Via SE37 instead
Class pool (K)	Global class	Via SE24 instead
Interface pool (J)	Global interface	Via SE24 instead
Subroutine pool (S)	External subroutines	Legacy

Status — set it to *Test program* for practice work. It is informational, but it signals intent to anyone browsing the system, and on real projects it keeps your experiments out of production program lists.

Leave these two ticked: *Unicode checks active* and *Fixed point arithmetic*. They are on by default in modern systems and turning them off causes subtle bugs.

E2. Packages, and Why \$TMP Matters

Simple definition: every repository object must live in a **package** — a folder that also decides whether the object can be transported to other systems.

Telugu: Program save chesinappudu SAP **Package** adugutundi. Rendu options: **\$TMP** (Local object) leda oka nijamaina Z package. Teda: **\$TMP lo unnavi vere system ki vellav** — nee machine lone untayi. Practice ki \$TMP sare, kaani nija project lo eppudu Z package ye.

After you save, SAP asks for a package. You have two answers:

Choice	Meaning	Consequence
\$TMP (or the Local Object button)	A temporary, personal package	Cannot be transported. Stays in this system forever. No transport request needed.
A Z package (e.g. <code>ZNV_PRACTICE</code>)	A real package	Transportable. SAP will ask for a transport request.

Which to use while learning:

- **Pure syntax practice** — `$TMP` is fine and saves you a step every time.
- **Anything you might want to keep, show, or move** — a real Z package.
- **Your December portfolio app — a real Z package, always.** Objects in `$TMP` cannot be transported, and building a portfolio piece somewhere it can never leave is a mistake you discover far too late.

Creating your own package: in SE80, choose *Package* from the dropdown, type `ZNV_PRACTICE` , press Enter, and confirm creation. It asks for a short description and a *software component* (`HOME` for customer development) and a transport layer. Do this once in your first week and put everything real in it.

Anchor to what you know: a package is a folder plus a deployment flag. `$TMP` is like working in a directory that is in `.gitignore` — fine for scratch work, useless for anything you need to ship.

E3. Transport Requests — How Code Leaves Your System

Simple definition: a **transport request** is a numbered container that records every change you make, so those changes can be moved to the test and production systems together.

Telugu: Nuvvu marchina prathi object oka **transport request** lo record avutundi. Aa request ni **release** cheste, adi DEV nunchi QA, taruvata PROD ki veltundi. Idi SAP lo **deployment** vidhaanam. Git lo commit + push laaga anukovachu — kaani ikkada okesari motham velthundi.

When you save an object in a real package, SAP shows the request dialog:

Prompt for Workbench request

Object

R3TR PROG ZNV_CARRIER_LIST

Request

S4HK900123

Short description

Practice reports – Nikhil

Own Requests

Create Request

✓

✗

S4H | 100

The transport request prompt. Pick an existing request from **Own Requests**, or make a new one with **Create Request**.

The two kinds of request:

Kind	Contains	Created from
Workbench request	Repository objects — programs, tables, classes, CDS views	Development work
Customizing request	Configuration settings, client-specific	IMG / SPRO configuration

The structure: a request contains **tasks**, one per developer. Your objects sit in your task; the request is released as a whole once every task inside it is released.

```

Request S4HK900123 "Purchase requisition app"
├── Task S4HK900124 (NIKHIL) → ZNV_CARRIER_LIST, ZPURCH_REQ
└── Task S4HK900125 (COLLEAGUE) → ZCL_PR_HANDLER
  
```

Releasing (in SE09): release your task first, then the request. Once released it moves toward QA. You cannot edit objects in a released request — you create a new one.

Anchor to what you know: a task is your commit, the request is the pull request, and releasing is the merge and deploy. The important difference is that SAP transports *objects in their current state*, not diffs — and there is no easy revert. That is why releasing to production is a controlled, scheduled activity everywhere.

Interview relevance: "Walk me through how your code gets to production" is a standard question. The answer is: create/attach a workbench request in DEV, release your task and request in SE09, basis imports it into QAS via STMS, it is tested, then imported into PRD. Saying that fluently marks you as someone who has seen a project.

E4. Write, Check, Activate, Run

Simple definition: four separate steps. Web development collapses them into one; ABAP does not, and skipping one is the most common beginner frustration.

Telugu: Naalugu **vere vere** adugulu: (1) raayadam, (2) **Save** Ctrl+S, (3) **Activate** Ctrl+F3, (4) **Run** F8. Chala mandi **Activate marchipotaru** — appudu "code marchanu kaani marpu raledu" ani confuse avutaru. Save ante database lo pettadam, Activate ante **run avvadaniki siddham cheyyadam**.

Step	Key	What it does	If you skip it
1. Write	—	Type the code	—

Step	Key	What it does	If you skip it
2. Syntax check	Ctrl+F2	Checks for errors without saving	You find errors later, more slowly
3. Save	Ctrl+S	Stores the source in the database	The code is lost
4. Activate	Ctrl+F3	Compiles it into a runnable version	Your changes do not take effect
5. Run	F8	Executes it	—

The trap: Save is not Activate. A saved-but-inactive program keeps running its *last activated* version. You change a line, save, run — and nothing changes. Every ABAP beginner loses time to this at least once. If your change did not take effect, press **Ctrl+F3** first.

Your first program — type this, do not paste it:

```
REPORT znv_carrier_list.

DATA: lt_carriers TYPE TABLE OF scarr,
      ls_carrier  TYPE scarr.

SELECT * FROM scarr INTO TABLE lt_carriers.

LOOP AT lt_carriers INTO ls_carrier.
  WRITE: / ls_carrier-carrid, ls_carrier-carrname, ls_carrier-currcode.
ENDLOOP.
```

SCARR is SAP's demo flight-data table and exists on virtually every training system, so this runs without you creating anything first.

Then rewrite it in modern ABAP — this is the version a 2026 interviewer wants to see:

```
REPORT znv_carrier_list.

SELECT carrid, carrname, currcode
  FROM scarr
 INTO TABLE @DATA(lt_carriers).

LOOP AT lt_carriers INTO DATA(ls_carrier).
  WRITE: / ls_carrier-carrid, ls_carrier-carrname, ls_carrier-currcode.
ENDLOOP.
```

What changed, and why it matters:

Old	Modern (7.4+)	Why
<code>SELECT *</code>	Named column list	Only fetch what you need

Old	Modern (7.4+)	Why
Separate <code>DATA:</code> declarations	<code>@DATA(...)</code> inline declaration	Less code, type derived automatically
No escape character	<code>@</code> before host variables	Required in modern Open SQL

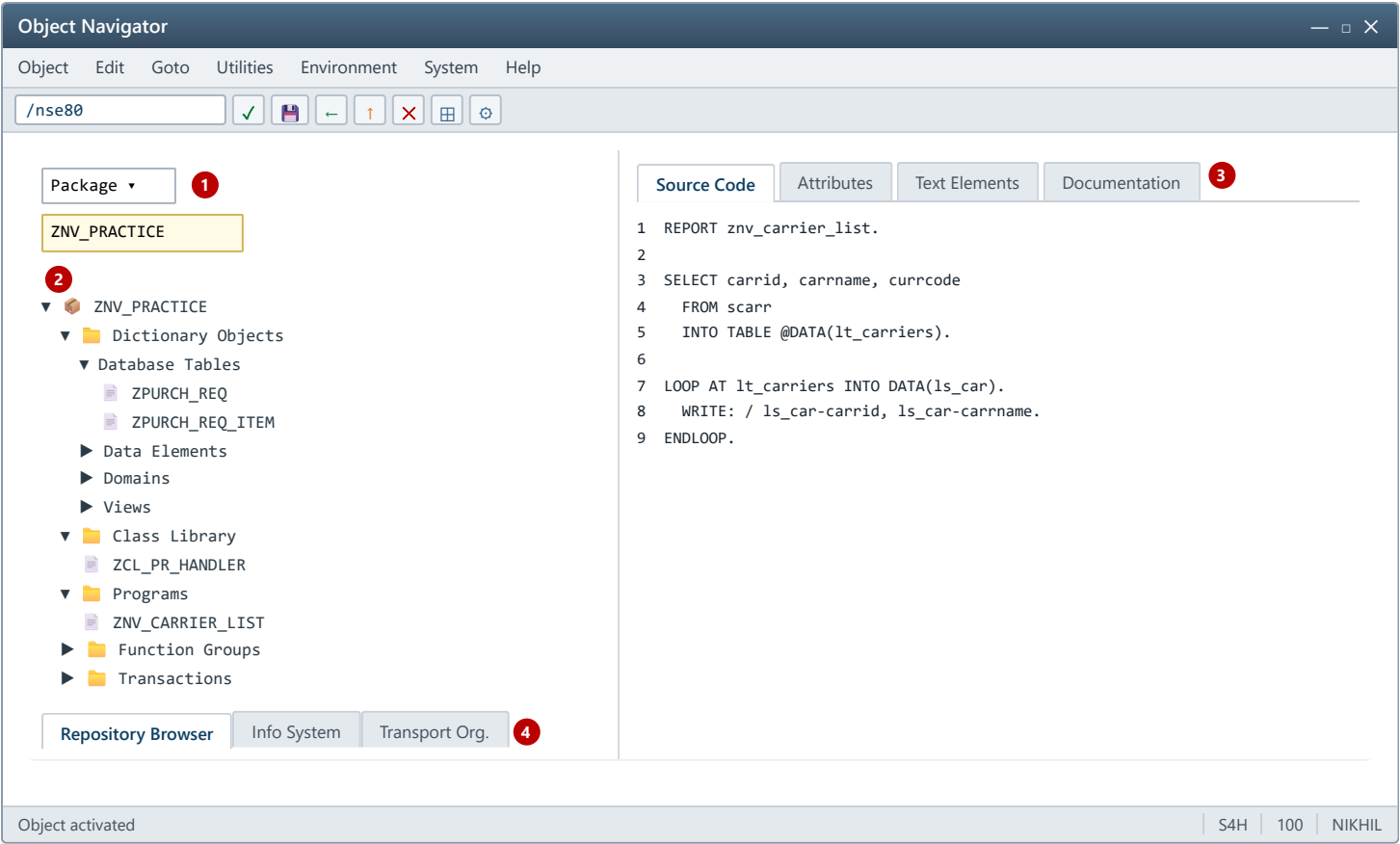
Write both versions in your practice slot this week. Being able to say "*I write modern ABAP, and here is the difference*" is a small, concrete thing that separates you from a candidate taught only the 2005 syntax — which, per Part B2 of your Launch Plan, is exactly the risk with institute courses.

Part F — SE80 and What Moved to Eclipse

F1. SE80 — The Object Navigator

Simple definition: SE80 is the one-window workbench. A navigation tree on the left, an editor on the right, and access to almost every object type SAP has — tables, programs, classes, function groups — without switching transactions.

Telugu: SE80 ante **anni okate chota**. Yeda vaipu tree, kudi vaipu editor. SE11, SE38, SE24, SE37 — ivi anni SE80 lopala nunche cheyyochu. **Package peru ista chaalu** — aa package lo unna anni objects tree lo kanapadutayi. Oka project ni ardham chesukovadaniki idi best tool.



SE80. **1** Object-type dropdown — Package, Program, Class, Function Group... **2** The tree of everything in that package. **3** Editor tabs for the selected object. **4** Browser tabs at the bottom left.

The object-type dropdown (callout 1) is what makes SE80 powerful. Choose what you want to browse by:

Choose	Then type	You get
Package	ZNV_PRACTICE	Everything in that package, grouped by type
Program	ZNV_CARRIER_LIST	One program with its includes and screens
Class/Interface	ZCL_PR_HANDLER	The class with its methods and attributes
Function Group	ZNV_FG01	The group with its function modules
Local Objects	—	Everything you put in \$TMP

Browsing by package is the skill worth building. When you join a project in January and someone says "the requisition enhancement is in package ZMM_PROC ", SE80 with that package name shows you the entire thing — every table, class, program and transaction — in one tree. That is how you understand unfamiliar code quickly, and it is much faster than opening objects one at a time.

The bottom-left tabs (callout 4):

Tab	Use
Repository Browser	The default tree

Tab	Use
Repository Information System	Search by object type, author, package, date (also SE84)
Transport Organizer	Your requests, inside SE80
Tag / MIME Repository	Web content — rarely needed

Useful habits in SE80:

- **Double-click any object** in the tree to open it on the right.
- **Right-click a package → Create** to add an object directly into it, so it lands in the right package automatically.
- **Where-used list** (Ctrl+Shift+F3) on any object shows everywhere it is referenced — indispensable before you change something.

Where-used is the one to remember. Before modifying a data element, a table or a method, run the where-used list. If it returns forty programs, your "small change" is not small. Interviewers ask about impact analysis; this is the tool that answers it.

F2. SE80 vs Eclipse (ADT) — the Line You Must Know

Simple definition: S/4HANA development is split. Classical objects can be built in SAP GUI or Eclipse. Modern objects — CDS, RAP, ABAP Cloud — can **only** be built in Eclipse.

Telugu: Idi ee doc lo **chala mukhyamaina point**. Purathana (classical) objects — tables, reports, classes — SE80 lo cheyyochu. Kaani **CDS views, RAP, ABAP Cloud** — ivi **Eclipse (ADT) lo matrame**. Nee December portfolio app motham CDS + RAP. Ante **Eclipse nerchukoka tappadu**. SE80 lo aa pani jarugadu.

Object	SE80 / SE11	Eclipse + ADT
Database tables, domains, data elements	✓	✓
Classical reports, includes	✓	✓
Classes, interfaces, function modules	✓	✓
Classical Dynpro screens	✓	✗
SAPscript / Smart Forms	✓	✗
CDS views	⚠ Display only	✓ Create and edit — only here
CDS annotations / metadata extensions	✗	✓
RAP behaviour definitions & implementations	✗	✓ Only here
Service definitions & bindings	✗	✓ Only here
ABAP Cloud development	✗	✓ Only here

What ADT is: *ABAP Development Tools* — a set of plugins for **Eclipse**. You install Eclipse, add the ADT plugins from SAP's update site, then create an "ABAP project" pointing at your system using the same client, user and password you use in SAP GUI.

What it feels like: a real IDE. Code completion, inline errors, quick fixes, refactoring, project search, Git-style comparison. After SE38, it is a noticeable relief — and it will feel familiar coming from VS Code.

Why SAP GUI is still worth all of Part A to E:

1. **Existing code lives there.** Every project has years of classical ABAP, and reading it is most of your first year.
2. **Configuration, debugging and monitoring are GUI-only.** ST22, SM37, SU53, SM30, SE93 — none of these have an ADT equivalent.
3. **Your FLT course teaches it,** and your practice system is a GUI system.
4. **Interviews test it.** "Which transaction shows short dumps?" is a real screening question.

The practical plan for you:

Now → December	Then
SAP GUI daily in the 9–11 slot — ABAP, DDIC, debugging, the fundamentals	Install Eclipse + ADT by early October , when the Fiori and RAP work starts

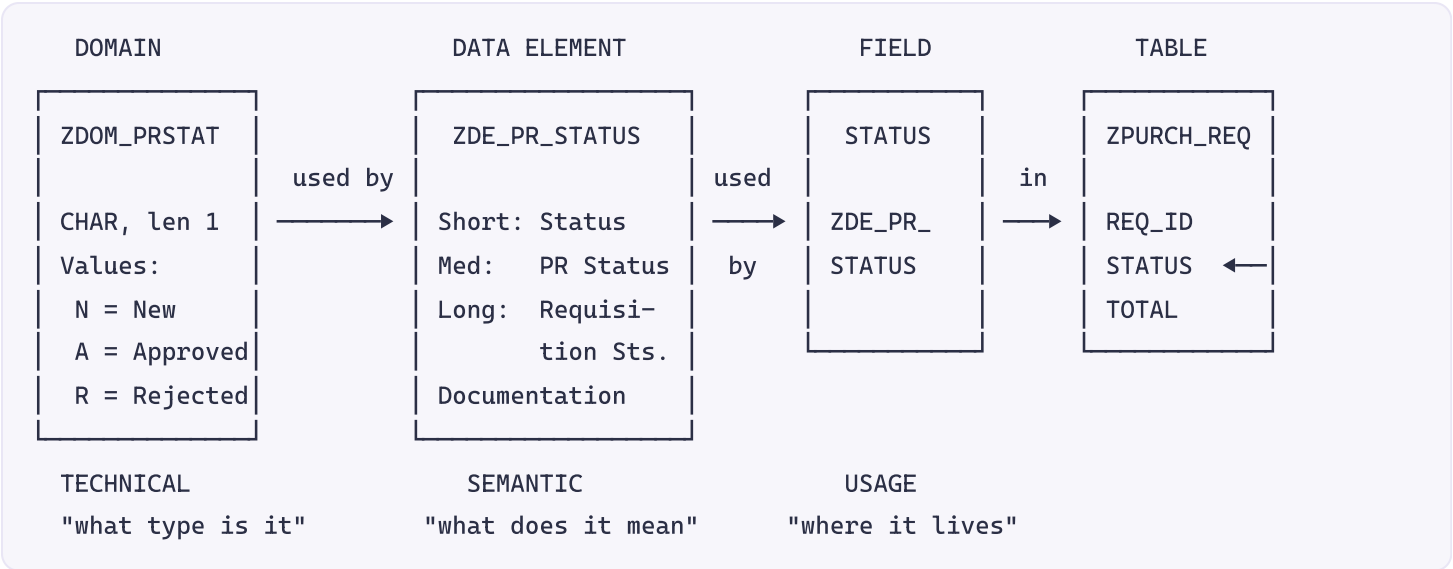
The honest warning, repeated because it decides your December: if your FLT system is a classical one without CDS and RAP support, you cannot build the portfolio app on it. Ask early, and if the answer is no, set up the free **BTP ABAP Environment trial** — that is what it is for.

Part G — The Data Model and SE11

G1. The SAP Data Model in One Picture

Simple definition: SAP does not let a table field simply be "CHAR 10". Every field points at a **data element** (what it *means*), which points at a **domain** (what it *technically is*). Three layers where SQL has one.

Telugu: SQL lo oka column ki direct ga `VARCHAR(10)` ani raastam. SAP lo **moodu layers** untayi: **Domain** (technical — type, length, valid values), **Data Element** (meaning — field label, documentation), inka **Field** (table lo actual column). Modata idi ekkuva anipistundi, kaani deeni valla **oke sari define chesi vandala tables lo vaadochu** — label marchali ante oke chota marchali.



Layer	Answers	Holds
Domain	What type is it, and which values are legal?	Data type, length, decimals, value range, value table, conversion routine
Data element	What does this mean to a user?	Field labels (short/medium/long/heading), documentation (the F1 help), search help
Field	Where is it used?	A column in a table or structure, typed by a data element

Why three layers instead of one — the argument that makes it click: define the domain `ZDOM_PRSTAT` once, and every status field in every table you ever build shares the same type, the same length and the same legal values. Change the label on the data element and **every screen in the system** that shows that field updates — because the F1 help and the field label are read from the data element, not hard-coded on the screen.

Anchor to what you know: the domain is a TypeScript type; the data element is that type plus i18n labels and documentation; the field is a variable declared with it. You already know why shared types beat repeating `string` everywhere — this is the same argument, applied across a system with 90,000 tables.

A practical shortcut you will see: a field can also use a **predefined type** (CHAR 10) directly with no data element. It works, and it is fine for a throwaway structure. **Do not do it in a real table** — you lose labels, F1 documentation and search help, and reviewers will flag it.

G2. The SE11 Initial Screen — Every Option

Simple definition: SE11's first screen is a list of radio buttons. Each one opens a different kind of dictionary object.

Telugu: SE11 modati screen lo **edu radio buttons** untayi. Prathi okkati vere rakam object teruchutundi. Nuvvu ekkuva vaade vi **Database table** inka **Domain / Data type**.

Radio button	Opens	You will use it
Database table ★	Transparent tables	Constantly
View	Database, projection, maintenance and help views	Occasionally — CDS replaces most of this
Data type ★	Sub-options: Data element , Structure , Table type	Constantly
Type Group	A set of related types (legacy)	Rarely
Domain ★	Technical type definitions	Often
Search help	F4 value help definitions	Sometimes
Lock object	Enqueue/dequeue for concurrent access	Sometimes

The three things under "Data type":

	What it is	Analogy
Data element	A single typed, labelled field	One typed variable
Structure	A group of fields with no database table behind it	An interface / object shape
Table type	A definition of an internal table of some row type	<code>Array<T></code>

The three buttons on the initial screen: *Display* (read-only), *Change* (edit), *Create* (new). You will spend most of your first months in **Display** — reading SAP's standard tables to understand what data exists.

A habit to build now: whenever you meet a new table name in code or in an F1 → Technical Information popup, open it in SE11 Display and read its fields. Ten minutes a day of this over four months is how people end up "just knowing" that `EKKO` is the purchase order header and `EKPO` its items.

G3. Domains — The Technical Layer

Simple definition: a **domain** defines the raw type of a field — its data type, length, and optionally the exact list of values it is allowed to hold.

Telugu: Domain ante field yokka **technical roopam** — e type, entha length, enni decimals, inka **e values allow chestamu**. Value range pettinappudu, aa list lo lenidi enter cheyyaleru — validation code raayakundane. Idi chala shakti vantidi.

The two tabs:

Definition tab — the type itself:

Field	Means
Data Type	CHAR, NUMC, DEC, CURR, QUAN, DATS, TIMS, INT4, LANG, CLNT, STRING...
No. of Characters	Length

Field	Means
Decimal Places	For DEC, CURR, QUAN
Output Length	How wide it displays
Convers. Routine	A conversion exit — e.g. <code>ALPHA</code> pads a number with leading zeros
Value Table	A table whose entries are the legal values

Value Range tab — the legal values:

Option	Use
Single values	A short fixed list: <code>N = New</code> , <code>A = Approved</code> , <code>R = Rejected</code>
Intervals	A numeric range
Value table	A table of legal entries, when the list is long or maintained by users

Fixed values give you three things for free: F4 help on every screen using the field, automatic input validation, and readable text instead of a bare code. No ABAP required.

The most common ABAP data types you will meet:

Type	Holds	Note
CHAR	Text, fixed length	Padded with spaces
NUMC	Digits only, stored as text	Keeps leading zeros — used for document numbers
STRING	Variable-length text	No padding
DATS	Date, 8 chars <code>YYYYMMDD</code>	Always this format internally
TIMS	Time, 6 chars <code>HHMMSS</code>	
DEC / CURR / QUAN	Packed decimals	CURR needs a currency field; QUAN needs a unit field
INT4	Integer	
CLNT	Client, 3 chars	The <code>MANDT</code> field
LANG	Language key	

The `ALPHA` conversion routine is worth understanding early. A NUMC(10) material number `100` is stored as `0000000100` but shown as `100` . When your SELECT returns nothing for a document number that clearly exists, the missing leading zeros are usually why. The fix is `CONVERSION_EXIT_ALPHA_INPUT` or the newer `|{ lv_num ALPHA = IN }|` .

G4. Data Elements — The Meaning Layer

Simple definition: a **data element** wraps a domain and adds what a *human* needs — the labels shown on screens and the documentation shown by F1.

Telugu: Data element ante domain ki **manishi ardham chesukune bhaagam** kalapadam — screen meeda kanapade **label**, inka F1 kotte vachhe **documentation**. Domain "CHAR 1" ani cheptundi; data element "idi Requisition Status" ani cheptundi.

The three tabs:

Data Type tab — what it is built on. Either an **elementary type** (a domain) or a **reference type**. Point it at your domain.

Field Label tab — four labels, and SAP picks the one that fits the space available:

Label	Length	Where it shows
Short	10 chars	Narrow columns
Medium	20 chars	Standard forms
Long	40 chars	Wide forms
Heading	55 chars	Report column headers

Fill in all four. Leaving them blank produces screens that show a field name instead of a label, which looks unfinished.

Further Characteristics tab — the search help attachment, and the *change document* flag if changes to this field should be logged.

Documentation — the part almost nobody fills in. Click *Documentation* and write what the field means. That text is exactly what appears when a user presses **F1** on the field. On real projects this is the difference between a system people can use and one where every field prompts a support ticket.

Naming conventions worth adopting now:

Object	Pattern	Example
Domain	Z<AREA>_<NAME>	ZPR_STATUS
Data element	ZDE_<NAME>	ZDE_PR_STATUS
Table	Z<AREA>_<NAME>	ZPURCH_REQ

Any consistent scheme works. Consistency is what gets noticed in a code review — not which scheme you picked.

G5. Creating a Z Table, Step by Step

Simple definition: the full walkthrough — from SE11's initial screen to an activated, usable table. This is the exercise your course will set, and the one you should be able to do without notes by October.

Telugu: Idi **full walkthrough** — SE11 nunchi modalu, activate varaku. Ee panini nuvvu **chusi kakunda** cheyyagalagali October ki. Rendu chotla kotta vaallu ekkuva tappu chestaru: (1) modati field **MANDT** pettadam marchipovadam, (2) **Technical Settings** nimpakapovadam — appudu activate avvadu.

Step 1 — Create. SE11 → *Database table* → **ZPURCH_REQ** → **Create**. Table names: **Z** or **Y** prefix, **maximum 16 characters**.

Step 2 — Short description. One line describing what it holds.

Step 3 — Delivery and Maintenance tab.

Field	Choose	Why
Delivery Class	A	Application table — master and transaction data. This is your default.
Data Browser / Table View Maint.	Display/Maintenance Allowed	Required if you want an SM30 maintenance screen later

The other delivery classes: **C** customizing, **L** temporary data, **G** protected customizing, **E** control table, **S** and **W** system tables. For custom application data, **A** is nearly always right.

Step 4 — Fields tab. The first field is always the client:

Dictionary: Maintain Table — ZPURCH_REQ						
Attributes Delivery and Maintenance Fields Entry help/check Currency/Quantity Fields						
Field	Key	Init	Data element	Type	Len	Short Description
MANDT	☒	☒	MANDT	CLNT	3	Client
REQ_ID	☒	☒	ZDE_PR_ID	CHAR	10	Requisition number
REQUESTER	☐	☐	ZDE_PR_USER	CHAR	12	Requested by
REQ_DATE	☐	☐	ZDE_PR_DATE	DATS	8	Requisition date
TOTAL_AMT	☐	☐	ZDE_PR_AMT	CURR	15	Total value
CURRENCY	☐	☐	WAERS	CUKY	5	Currency
STATUS	☐	☐	ZDE_PR_STATUS	CHAR	1	Status

The **Fields** tab. **MANDT** is always the first key field. The  icon in the toolbar opens Technical Settings;  activates.

The rules for this tab:

- **MANDT first, always** — key + initial, data element **MANDT**. Without it your table is client-independent and every client sees the same rows. This is the single most common beginner mistake.
- **Key fields must come first**, contiguously, before any non-key field.
- Tick **Initial Values** on key fields.
- Give each field a **data element**, not a predefined type.

Step 5 — Currency/Quantity Fields tab. If you used a **CURR** or **QUAN** field, this tab is **mandatory**. A currency amount is meaningless without a currency, so SAP forces you to say which field holds it:

Field	Reference table	Reference field
TOTAL_AMT	ZPURCH_REQ	CURRENCY

Skip this and activation fails with an error that does not obviously point here. It is the second most common beginner mistake.

Step 6 — Technical Settings. Click the  icon (or *Goto* → *Technical Settings*):

Setting	Choose	Meaning
Data Class	APPL0 master / APPL1 transaction / APPL2 customizing	Which tablespace it lives in
Size Category	0–9	Expected number of rows; SAP sizes the initial extent from it
Buffering	Not allowed / allowed but off / switched on	Whether rows are cached on the app server

Buffering guidance: switch it on only for small, rarely-changed tables — configuration and lookup data. Never buffer a table that is written constantly; you get stale reads. For a transactional table like this one, leave buffering off.

Step 7 — Enhancement Category. *Extras* → *Enhancement Category* → usually "**Can be enhanced (deep)**". Skipping it produces a warning on activation.

Step 8 — Indexes (optional). *Goto* → *Indexes* to add a secondary index if you will frequently select on non-key fields. Same trade-off as any database: faster reads, slower writes.

Step 9 — Activate (Ctrl+F3). This creates the actual database table. The status must read *Active*.

Then check your work: open SE16N on ZPURCH_REQ. An empty result list with your column headings means the table exists and is correct.

G6. Views, Search Helps and Lock Objects

Simple definition: the remaining three SE11 object types — combining tables, providing F4 help, and preventing two users editing the same record.

Telugu: SE11 lo migilina moodu: **View** (tables kalipi chudadam), **Search help** (F4 kotte vachhe list), **Lock object** (iddaru okesari oke record marchakunda aapadam). Ivi roju vaadaru, kaani interviews lo adugutaru — muriyu ardham chesukovadam sulabham.

Views — combining tables

View type	What it does	Still used?
Database view	An inner join across tables, defined in DDIC	Yes, but CDS views replace it for new work
Projection view	A subset of one table's fields	Occasionally

View type	What it does	Still used?
Maintenance view	Lets users maintain several related tables together via SM30	Yes
Help view	Used inside a search help	Rarely built by hand

The important context for 2026: for new development, **CDS views** (built in Eclipse, Part F2) have superseded classical database views. They do everything a database view does plus associations, annotations, expressions and parameters — and they are what RAP is built on. Learn classical views enough to *read* them; build CDS views for anything new.

Search helps — the F4 popup

A **search help** defines what appears when a user presses F4 on a field.

Kind	What it is
Elementary	One search path — a table or view, with selection and display fields
Collective	Several elementary search helps offered as tabs in one popup

How it gets attached: to a data element (so every field using it inherits the help), to a table field, or to a screen field directly. Attaching at the **data element** level is best — define once, works everywhere.

The simplest alternative: if your domain has **fixed values** (G3), you get F4 help automatically with no search help needed. Use that whenever the list is short and static.

Lock objects — preventing collisions

The problem: two users open the same requisition and both save. Without locking, one silently overwrites the other.

The mechanism: create a lock object named `EZPURCH_REQ` (lock objects start with **E**) over your table. Activating it generates two function modules automatically:

Generated	Call it
<code>ENQUEUE_EZPURCH_REQ</code>	Before you start changing a record
<code>DEQUEUE_EZPURCH_REQ</code>	After you save or cancel

Lock mode	Meaning
E — Exclusive	One user only; the standard choice
S — Shared	Many readers, no writer
X — Exclusive, non-cumulative	Cannot be requested twice, even by the same program

Where you will meet locks in practice: the message *"Object is locked by user X"*. Transaction **SM12** shows current locks and who holds them — including the case from Part C3 where you have locked something against

yourself in another session.

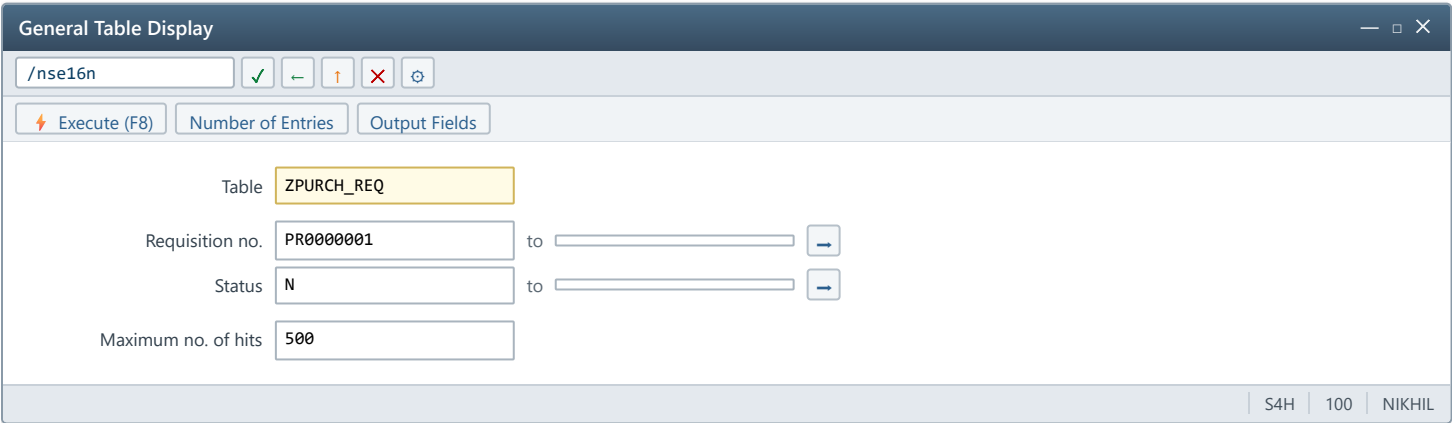
Anchor to what you know: this is pessimistic locking, the same idea as `SELECT ... FOR UPDATE` in SQL, or the version-column optimistic locking you met in the REST API notes. SAP defaults to pessimistic locking because a user may sit on an edit screen for ten minutes.

Part H — Reading Data, Debugging, and Your First Week

H1. SE16N — Reading Table Data Properly

Simple definition: SE16N shows you the contents of any table. It is the transaction you will open most often after SE11, and using it well is a genuine time-saver.

Telugu: SE16N tho e table lo aina data chudochu. SE11 tarvaata nuvvu ekkuva vaade T-code idhe. Chala mandi kevalam table peru ista, Execute kottutaru. Kaani **selection fields, output fields, sorting, export** — ivi telisthe pani chala vegam avutundi.



SE16N's selection screen. Each field takes a range (from-to) and the → button opens multiple selection — several values, exclusions, patterns.

Things worth knowing that most people never use:

Feature	How	Why
Multiple selection (the → button)	Click the arrow beside any field	Enter several single values, ranges, or <i>exclude</i> values
Wildcards	* and + in a text field	PR* finds everything starting with PR
Number of Entries	Button on the toolbar	Counts rows without fetching them — fast on huge tables
Output Fields	Button on the toolbar	Choose which columns to show; huge tables become readable

Feature	How	Why
Sort and filter	ALV toolbar on the result list	Same grid controls as everywhere in SAP
Export	List → Export → Spreadsheet	Get the data into Excel
Layouts	Save your column selection	Reuse it tomorrow instead of rebuilding

Always set a maximum hit count. The default is usually 500. Removing it on a table with fifty million rows produces a `TIME_OUT` dump and, on a shared system, an annoyed basis team.

SE16 vs SE16N vs SE16H:

Note	
SE16	The classic Data Browser. Present on every system, including very old ones.
SE16N ★	More flexible selection and output. Your default.
SE16H	S/4HANA only — adds grouping, aggregation and totals, pushed down to HANA

Try SE16H once this week if your system is S/4HANA. Being able to say *"I used SE16H to aggregate in the database rather than pulling rows"* connects directly to the code-to-data principle that CDS and HANA are built on.

H2. SM30 and the Table Maintenance Generator

Simple definition: your Z table has no user interface. The **Table Maintenance Generator** creates one automatically, and users then reach it through SM30.

Telugu: Nuvvu Z table kattavu — kaani andulo data pettadaniki users ki **screen ledu. Table Maintenance Generator** aa screen ni automatic ga tayar chestundi. Tarvaata users **SM30** lo aa table peru isthe, data add/edit/delete cheyyochu. Idi oke sari cheyyali, code raayanasaram led.

Generating it — from inside SE11 with your table open:

Utilities → Table Maintenance Generator

Field	Enter	Note
Authorization Group	<code>&NC&</code> or a real group	<code>&NC&</code> means "no group" — fine for practice
Function Group	e.g. <code>ZNV_PR_MAINT</code>	Created if it does not exist
Maintenance type	One step (or two step)	One step = a single editable list. Start here.
Overview screen	e.g. <code>0001</code>	Any free screen number

Then click **Create**. SAP generates the screens and the function group for you.

Prerequisite: the table's *Data Browser/Table View Maint.* setting (Part G5, step 3) must be **Display/Maintenance Allowed**. If it is not, the generator refuses.

Using it: `/nSM30` → enter `ZPURCH_REQ` → **Maintain**. You get an editable list with New Entries, Save, and Delete.

One step vs two step:

Type	Screens	Use when
One step	A single overview list, editable in place	Few fields — your default
Two step	An overview list plus a detail screen per row	Many fields per record

SM34 — view clusters group several related maintenance views into one guided sequence (header, then items, then texts). You will meet it on real projects; you rarely build one early on.

Why this matters beyond the mechanics: the December portfolio app needs a way to get test data into your tables. SM30 is the two-minute answer, and it lets you focus your effort on the CDS and RAP layers rather than on building a data-entry screen.

H3. The ABAP Debugger

Simple definition: the tool that stops a running program at a chosen line so you can inspect every variable and step through the logic. Per your Launch Plan, this is the skill that separates you fastest — most of a consultant's job is reading running code, not writing new code.

Telugu: Debugger ante — program madhyalo **aapi**, prathi variable lo em undo chudadam, line by line munduku vellatam. Nija SAP job lo **kotta code raayadam kanna, unna code lo tappu vetakadam ekkuva**. Idi Chrome DevTools laage — nuvvu ade already chesavu, kevalam keys veru.

Three ways to start it:

Method	How	Use when
<code>/h</code>	Type <code>/h</code> , Enter, then do the action	You are inside a transaction and want to stop at the next step
Session breakpoint	In the editor, click the stop icon beside a line (or Ctrl+Shift+F12)	You know exactly which line
<code>BREAK-POINT.</code>	Write the statement in the code	Temporary; never leave it in transported code

The step controls — the four keys that are the whole tool:

Key	Name	Does
F5	Step into	Execute one line; go <i>inside</i> a call
F6	Execute	Execute one line; step <i>over</i> a call

Key	Name	Does
F7	Return	Run to the end of the current routine and come back out
F8	Continue	Run on until the next breakpoint or the end

Anchor to what you know: these are exactly Chrome DevTools' step-into, step-over, step-out and resume. Same four operations, same mental model — only the keys differ.

The debugger's desktops (tabs across the top):

Tab	Shows
Standard	Source code plus a small variable watch list
Structures	A structure's fields expanded
Tables	Internal table contents, row by row — the one you will use most
Objects	Object instances and their attributes
Break./Watchpoints	Everything currently set

Watching a variable: type its name into the variable column and its value appears beside it. Double-click an internal table to open it in the Tables tab and see every row.

Watchpoints — the underused feature. A breakpoint stops at a *line*; a **watchpoint** stops when a *variable changes value*. When a field ends up wrong and you have no idea which of two hundred lines did it, set a watchpoint on that field and run. The debugger stops exactly at the guilty statement.

Changing values at runtime. In change mode you can edit a variable mid-execution — useful for testing a branch you cannot otherwise reach. It requires debug-change authorization and is normally forbidden in production, for obvious reasons.

The weekly drill, from your Launch Plan: take code you wrote, introduce one bug on purpose, wait a day, then find it using **only** the debugger. Do this every Friday morning in your 9–11 slot. By December, "*walk me through how you would debug this*" becomes a question you look forward to.

H4. Your First Week, and the Keys Worth Memorising

Simple definition: the closing checklist — what to do in your practice slot this week, and the shortcuts that pay for themselves fastest.

Telugu: Ee week nee 9–11 slot lo cheyyalsina panula list. Prathi okkati chinna pani, kaani anni kalisi neeku SAP GUI meeda **manchi pattu** istayi. Chivarilo unna keyboard shortcuts — avi **gurthupettukunte** nee vegam rettimpu avutundi.

This week's checklist

- ☐ Log in and note your **system ID and client** from the status bar
- ☐ Open **System → Status** and **write down the S/4HANA release** — you will be asked this in interviews

- ☐ Set your date format in **SU3**
- ☐ Turn on "**Show keys within dropdown lists**" (Alt+F12 → Options → Interaction Design)
- ☐ Set a **colour for this system** so you never confuse it with another
- ☐ Add ten transactions to **Favorites** (Part C2)
- ☐ Press **F1** → **Technical Information** on five fields in ME23N or VA03; note the table names
- ☐ Run `SEARCH_SAP_MENU` once, to know it exists
- ☐ Open **ABAPDOCU** and run one example program
- ☐ Create `ZNV_CARRIER_LIST` in SE38 and run it (Part E4)
- ☐ Rewrite it using inline declarations and a named column list
- ☐ Cause a divide-by-zero dump, then find it in **ST22** and read all five sections
- ☐ Create your **Z package**, and put everything real in it from now on
- ☐ Ask FLT: **what release is this system, does it support CDS and RAP, and does my access continue after December?**

That last item is the most important line on the page. It determines whether your December portfolio app can be built where you are already practising, or whether you need the free BTP trial as well.

The keyboard shortcuts that matter

Key	Does	Where
Enter	Process the screen	Everywhere
Ctrl+S	Save	Everywhere
F3	Back	Everywhere
Shift+F3	Exit transaction	Everywhere
F12	Cancel	Everywhere
Ctrl+ /	Jump to the command field	Everywhere
F1	Field help → Technical Information	Any field
F4	Value help	Any field
F8	Execute	Reports, selection screens
Ctrl+F2	Syntax check	Editors
Ctrl+F3	Activate	Editors, SE11
Ctrl+Shift+F3	Where-used list	Any object
Ctrl+F	Find	Lists, code
F5 / F6 / F7 / F8	Debugger: into / over / out / continue	Debugger
Alt+F12	Customize local layout	Everywhere

Three to learn first, if you learn nothing else: Ctrl+/ (go anywhere), **F1** → **Technical Information** (find the table behind any field), and **Ctrl+F3** (activate — the step everyone forgets).

Where this fits in the bigger plan

SAP GUI is the ground floor. It is where you will spend your 9–11 slot until December, where your FLT course lives, and where you will read other people's classical ABAP for your entire first year on a project.

But remember the line from Part F2: **the modern half of your target stack is not built here**. CDS views, RAP behaviour definitions and service bindings all live in Eclipse with ADT, and your December portfolio app depends on them. SAP GUI gets you fluent in the fundamentals and employable in the interview room. Eclipse is where October onward happens.

Get fast here first. Everything else is easier afterwards.
